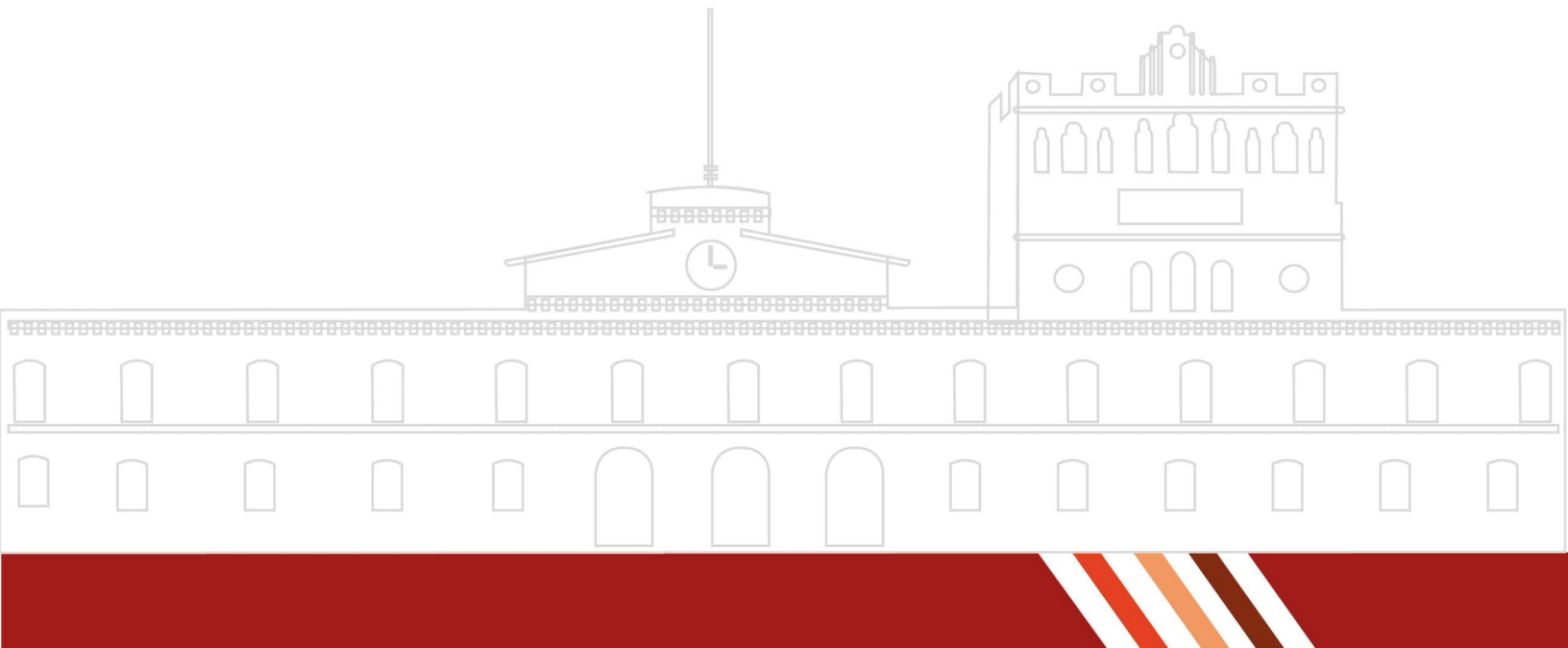


REPORTE DE PRÁCTICA NO. 6

Práctica 6 Análisis sintáctico de expresiones aritméticas

ALUMNOS:

Gutiérrez Reyes Jasson Axel
Pérez García Alejandro Máximo
Sánchez Barrón Javier Alejandro
Zúñiga Montoya José Antonio



Contents

1	Introducción	3
2	Marco Teórico	3
3	Objetivos	3
3.1	Objetivo General	3
3.2	Objetivos Específicos	3
4	Desarrollo	3
4.1	Librerías y Cabeceras Utilizadas	3
4.2	Analizador Léxico (lexico.l)	4
4.3	Analizador Sintáctico (sintactico.y)	5
5	Resultados	6
6	Conclusiones	6
7	Cuestionario	6
8	Bibliografía	7

Introducción

El análisis sintáctico es la segunda fase del proceso de compilación. Mientras que el analizador léxico se encarga de reconocer componentes aislados (tokens), el analizador sintáctico agrupa estos tokens para formar estructuras mayores y verificar que cumplan con las reglas gramaticales del lenguaje. En esta práctica, se utilizan de manera conjunta las herramientas Lex y Yacc (Bison) para construir un programa capaz de validar expresiones aritméticas, respetando la jerarquía de operadores y detectando errores de construcción en el código fuente.

Marco Teórico

El analizador sintáctico (Parser) es un programa que analiza una cadena de tokens para determinar su estructura gramatical utilizando una Gramática Libre de Contexto (GLC). Si la estructura es válida, construye lógicamente un árbol sintáctico; si es inválida, emite mensajes de error.

Para la generación automática de estos analizadores se utilizan herramientas como **Yacc / Bison**. Estos programas reciben como entrada un archivo con extensión `.y` que contiene la especificación de una gramática en formato BNF (Backus-Naur Form) y generan código en C que implementa un analizador sintáctico ascendente (LALR). Un aspecto crítico en este diseño es la definición de la **Asociatividad y Precedencia**, reglas matemáticas que resuelven ambigüedades indicando, por ejemplo, que la multiplicación debe evaluarse antes que la suma.

Objetivos

Objetivo General

Desarrollar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar operaciones aritméticas, asegurando la detección de errores y la correcta validación de las gramáticas definidas.

Objetivos Específicos

- Diseñar un programa en Lex que identifique números enteros y números decimales, así como símbolos que representan las operaciones de suma, resta, multiplicación y división.
- Implementar un programa en Yacc que verifique la correcta construcción de las operaciones aritméticas mediante una gramática formal.

Desarrollo

El proyecto se dividió en la construcción de dos módulos independientes que se comunican entre sí. Primero, se desarrolló el analizador léxico para reconocer los componentes y enviarlos mediante la instrucción `return`. Posteriormente, se estructuró la gramática en Yacc declarando la precedencia de los operadores.

Librerías y Cabeceras Utilizadas

Para el correcto funcionamiento y comunicación entre los analizadores generados por Lex y Yacc, así como para la interacción con el sistema operativo, se incluyeron las siguientes directivas de preprocesador en lenguaje C:

- `<stdio.h>` (**Standard Input/Output**): Es la librería estándar de entrada y salida de C. En el analizador sintáctico (`sintactico.y`), se utiliza fundamentalmente para dos tareas: permitir el uso de la función `printf` para imprimir en la consola los mensajes de éxito o error, y habilitar el manejo de

archivos mediante la estructura `FILE` y la función `fopen()`, lo que permite redirigir el código fuente de los archivos de prueba hacia el flujo de lectura del compilador.

- **<stdlib.h> (Standard Library):** Contiene funciones de utilidad general del lenguaje C, como la gestión dinámica de memoria. Aunque en nuestro código de usuario no llamamos explícitamente a funciones como `malloc` o `free`, es obligatorio incluirla porque el código interno en C que generan automáticamente Lex y Yacc depende profundamente de esta librería para gestionar los búferes de memoria donde almacenan los tokens.
- **"y.tab.h" (Cabecera Dinámica de Yacc):** A diferencia de las librerías estándar que se encierran entre signos de menor y mayor que (< >), esta cabecera se encierra entre comillas dobles para indicarle al compilador que debe buscar el archivo en la misma carpeta del proyecto. Es generado automáticamente por Yacc y contiene las definiciones numéricas de todos los `%token` declarados en la gramática. Su inclusión dentro de `lexico.l` es el puente de comunicación vital: es lo que permite que Lex entienda qué significa la instrucción `return NUMERO`; y le envíe la señal correcta al analizador sintáctico.

Declaración de Variables y Funciones Externas: Además de las librerías, en la sección de definiciones de Yacc (`sintactico.y`) es obligatorio declarar el enlace con el analizador léxico mediante las siguientes sentencias:

- `extern FILE *yyin;`
La palabra reservada `extern` le indica al compilador de C que esta variable global ya fue creada en otro archivo (específicamente en el código generado por Lex). `yyin` es el puntero de archivo que Lex utiliza por defecto para leer los caracteres. Al "llamarla" desde Yacc, el método `main` adquiere el poder de manipularla, permitiéndonos asignarle el archivo `.txt` que el usuario pasa por la consola antes de iniciar el análisis.
- `extern int yylex();`
De manera similar a `yyin`, esta línea declara la existencia de la función principal del escáner generada por Lex. Yacc necesita saber que esta función existe y que devuelve un número entero (el ID del token). Internamente, la función `yparse()` de Yacc ejecutará `yylex()` repetidamente cada vez que necesite consumir una nueva palabra del archivo.
- `void yyerror(const char *s);`
Es la declaración (prototipo) de la función de manejo de errores de Yacc. Cuando el analizador sintáctico detecta que la secuencia de tokens no coincide con ninguna regla gramatical (por ejemplo, al encontrar dos signos de suma juntos), detiene su proceso normal y llama automáticamente a esta función, pasándole un mensaje de error por defecto (como "syntax error"). Declararla arriba asegura que el compilador de C reconozca la función antes de que `yparse()` intente usarla.

Analizador Léxico (lexico.l)

```
%{
#include "y.tab.h"
#include <stdlib.h>
}%
%option noyywrap
%%
[0-9]+(\.[0-9]+)?      { return NUMERO; }
[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
"==" | "!=" | "<=" | ">=" | "<" | ">" { return CONDICIONAL; }
"+"                   { return SUMA; }
"-"                   { return RESTA; }
"*"                   { return MULTIPLICACION; }
```

```

"/"          { return DIVISION; }
"("          { return PARENTESIS_IZQ; }
")"          { return PARENTESIS_DER; }
\n           { return SALTO; }
[ \t]+       { /* Ignoramos espacios en blanco */ }
.            { return ERROR_LEXICO; }
%%

```

Analizador Sintáctico (sintactico.y)

```

%{
#include <stdio.h>
#include <stdlib.h>
extern FILE *yyin;
extern int yylex();
void yyerror(const char *s);
int linea = 1;
%}
%token NUMERO IDENTIFICADOR CONDICIONAL
%token SUMA RESTA MULTIPLICACION DIVISION PARENTESIS_IZQ PARENTESIS_DER SALTO ERROR_LEXICO
%left SUMA RESTA
%left MULTIPLICACION DIVISION
%%
entradas: /* Vacio */ | entradas linea_codigo ;
linea_codigo:
    SALTO { linea++; }
    | expresion SALTO { printf("[Linea-%d]-EXITOSO.\n", linea); linea++; }
    | error SALTO { yyerrok; printf("[Linea-%d]-ERROR-SINTACTICO.\n", linea); linea++; }
    ;
expresion:
    NUMERO | IDENTIFICADOR
    | expresion SUMA expresion
    | expresion RESTA expresion
    | expresion MULTIPLICACION expresion
    | expresion DIVISION expresion
    | PARENTESIS_IZQ expresion PARENTESIS_DER
    ;
%%
void yyerror(const char *s) {}
int main(int argc, char **argv) {
    if (argc > 1) { yyin = fopen(argv[1], "r"); }
    yyparse();
    return 0;
}

```

Resultados

Se crearon dos archivos de texto para someter el analizador a pruebas. La ejecución mediante `gcc lex.yy.c y.tab.c -o mi_parser.exe` arrojó los siguientes resultados desde la línea de comandos:

```
C:\Flex Windows\EditPlusPortable\Lex>mi_parser.exe correcto.txt
--- INICIANDO ANALISIS SINTACTICO ---
[Línea 1] EXITOSO: Expresion Aritmetica Valida.
[Línea 2] EXITOSO: Expresion Aritmetica Valida.
--- ANALISIS FINALIZADO ---

C:\Flex Windows\EditPlusPortable\Lex>mi_parser.exe incorrecto.txt
--- INICIANDO ANALISIS SINTACTICO ---
[Línea 1] --- ERROR SINTACTICO DETECTADO ---
[Línea 2] --- ERROR SINTACTICO DETECTADO ---
--- ANALISIS FINALIZADO ---
```

Figure 1: Ejecución del programa.

El analizador rechazó correctamente las cadenas mal formadas (como operadores adyacentes o paréntesis desbalanceados) y se recuperó del error para seguir analizando gracias a la instrucción `yyerror`.

txt con expresiones correctas:

```
12 + 5 * 3
(4.5 - 2) / 10
variable1 * 2
```

txt con expresiones incorrectas:

```
12 + * 5
(4.5 - 2
variable1 / / 2
```

Conclusiones

La integración de Lex y Yacc abstrae la complejidad iterativa requerida para la validación de estructuras lógicas. En lugar de desarrollar algoritmos manuales de verificación de jerarquía matemática, Yacc permite definir la estructura de manera puramente declarativa utilizando gramáticas formales. El manejo de errores integrado demuestra la capacidad de crear compiladores resilientes que no detienen su ejecución ante el primer error encontrado en el código fuente.

Cuestionario

1. **¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?**
Se detectaron errores de estructura, específicamente la colocación consecutiva de dos operadores aritméticos y la falta de un paréntesis de cierre. La gramática definida en Yacc identificó esto porque en sus reglas de producción (BNF) no existe ninguna derivación que contemple un patrón como `expresion SUMA MULTIPLICACION expresion`. Al no encontrar un camino válido, rechaza la instrucción.
2. **¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar las operaciones y las prioridades de las operaciones?**

Contribuye a resolver la ambigüedad matemática. Al usar la directiva `%left`, se agrupan primero los operadores de la izquierda ante igual jerarquía. Además, el orden de declaración en Yacc asegura que las operaciones de mayor prioridad (multiplicación y división) se resuelvan en niveles más profundos del árbol sintáctico.

3. **¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como $(a + b)^2$, raíz cuadrada, así como operaciones trigonométricas?**

En Lex, se deben agregar expresiones regulares para reconocer lexemas como `^`, `sqrt`, `sin` y devolver sus tokens correspondientes (`POTENCIA`, `RAIZ`, `SENO`). En Yacc, se deben registrar estos tokens, definir su precedencia (la potencia con `%right`) y agregar nuevas producciones BNF a la regla `expresion`, por ejemplo: `expresion POTENCIA expresion`.

Bibliografía

- Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de autómatas. Editorial Alfaomega. Argentina.
- Gutú, O. (2013). Primer curso en teoría de autómatas y lenguajes formales.
- Pearson Educación. México. Hopcroft, J. E. Motwani, R. & Ullman, J. D. (2007). Introducción a la teoría de automatas, lenguajes y computación. Pearson Educación. México.